

Mutation Testing, 다시 보기

어떤 mutant를 고르고, 시간이 지나면 어떻게 변하나?

Blind spots in mutant selection · Longitudinal mutant killability

⇒ 현재 mutation testing의 한계와 확장 방향

뮤테이션 테스트...

1. 뮤테이션 타겟을 정함.
2. 뮤턴트를 많이 생성. 그 후, 각 뮤턴트에 대해:
 - a. 현재 Test Suite로 탐지(Kill) 가능한가? : 탐지 (Kill), 불탐 (Survive)
3. Mutation Score를 계산
 - a. $\# \text{ of killed mutants} / \# \text{ of mutants}$: $\uparrow$ better
4. 위 mutation score를 기반으로 현재 test suite 평가 (또는 다양한 디버깅에 사용)

뮤테이션 테스트...

1. 뮤테이션 타겟을 정함.
2. 뮤턴트를 많이 생성. 그 후, 각 뮤턴트에 대해:
 - a. 현재 Test Suite로 탐지(Kill) 가능한가? : 탐지 (Kill), 불탐 (Survive)
3. Mutation Score를 계산
 - a. $\# \text{ of killed mutants} / \# \text{ of mutants}$: $\uparrow$ better
4. 위 mutation score를 기반으로 현재 test suite 평가 (또는 다양한 디버깅에 사용)

뮤테이션 테스트...

1. 뮤테이션 타겟을 정함.
2. 뮤테트를 많이 생성. 테스트할 뮤테트를 선택(mutant selection) 또는 실행 우선순위 부여 (mutant prioritization). 그 후, 각 뮤테트에 대해:
 - a. 현재 Test Suite로 탐지(Kill) 가능한가? : 탐지 (Kill), 불탐 (Survive)
3. Mutation Score를 계산
 - a. $\# \text{ of killed mutants} / \# \text{ of mutants}$: $\uparrow$ better
4. 위 mutation score를 기반으로 현재 test suite 평가 (또는 다양한 디버깅에 사용)

Mutant Selection

- 각 뮤텐트에 대해 **utility score**를 부여, 이를 기준으로 모든 생성된 뮤텐트들을 정렬, 그 후 top k 선택.
- Learning-based methods: predict utility scores (e.g., FaRM: fault-revealing [1], Cerebro: subsuming [2])

[1] T. Titcheu Chekam, M. Papadakis, T. F. Bissyande, Y. Le Traon, and K. Sen, "Selecting fault revealing mutants," Empirical Software Engineering, vol. 25, no. 1, pp. 434–487, 2020

[2] A. Garg, M. Ojdanic, R. Degiovanni, T. Titcheu Chekam, M. Papadakis, and Y. Le Traon, "Cerebro: Static subsuming mutant selection," IEEE Transactions on Software Engineering, vol. 49, no. 1, pp. 24–43, 2023

Mutant Selection

- 각 뮤턴트에 대해 **utility score**를 부여, 이를 기준으로 모든 생성된 뮤턴트들을 정렬, 그 후 top k 선택.
- Learning-based methods: predict utility scores (e.g., FaRM: fault-revealing [1], Cerebro: subsuming [2])

문제점: 선택된 뮤턴트들이 같은/비슷한 code context와 mutation operator 패턴을 공유

[1] T. Titcheu Chekam, M. Papadakis, T. F. Bissyande, Y. Le Traon, and K. Sen, "Selecting fault revealing mutants," Empirical Software Engineering, vol. 25, no. 1, pp. 434–487, 2020

[2] A. Garg, M. Ojdanic, R. Degiovanni, T. Titcheu Chekam, M. Papadakis, and Y. Le Traon, "Cerebro: Static subsuming mutant selection," IEEE Transactions on Software Engineering, vol. 49, no. 1, pp. 24–43, 2023

Mutant Selection

```
int calculate_fee(int age, int member, int amount) {  
    if (age < 18 || member == 1 || amount > 100) {  
        return amount * 0.8;  
    }  
  
    if (amount > 50) {  
        return amount * 0.9;  
    }  
  
    return amount;  
}
```

mutant	location	mutation	Score
m1	Line 2	age < 18 → age <= 18	0.97
m2	Line 2	age < 18 → age >= 18	0.95
m3	Line 2	first → &&	0.94
m4	Line 2	member == 1 → member != 1	0.93
m5	Line 2	second → &&	0.92
m6	Line 2	amount > 100 → amount >= 100	0.91
m7	Line 6	amount > 50 → amount >= 50	0.87
m8	Line 7	0.9 → 1.0	0.84
m9	Line 10	return amount → return 0	0.81

Mutant Selection

```
int calculate_fee(int age, int member, int amount) {  
    if (age < 18 || member == 1 || amount > 100) {  
        return amount * 0.8;  
    }  
  
    if (amount > 50) {  
        return amount * 0.9;  
    }  
  
    return amount;  
}
```

mutant	location	mutation	Score
m1	Line 2	age < 18 → age <= 18	0.97
m2	Line 2	age < 18 → age >= 18	0.95
m3	Line 2	first → &&	0.94
m4	Line 2	member == 1 → member != 1	0.93
m5	Line 2	second → &&	0.92
m6	Line 2	amount > 100 → amount >= 100	0.91
m7	Line 6	amount > 50 → amount >= 50	0.87
m8	Line 7	0.9 → 1.0	0.84
m9	Line 10	return amount → return 0	0.81

$k = 5 \rightarrow \{m1, m2, m3, m4, m5\}$

Structural Concentration $\Rightarrow$ *blind spots*.

LinePool (LinePool(s_L , s_W))

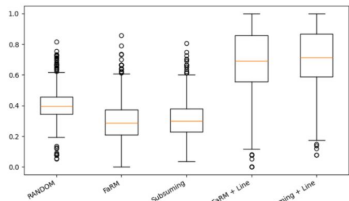
1. 같은 라인에 생성된 뮤탄트들을 그룹화: $G_i = \{m \mid m \text{ is located on line } l_i\}$
2. 라인 그룹들(G_i)을 정렬: $\text{score}(G_i) = \max(\{\text{score}(m) \mid m \text{ in } G_i\})$
3. 같은 그룹 내 뮤탄트들을 정렬
4. Round-robin으로 뮤탄트들을 선택

Round 1: best mutant from line A, line B, line C, ...

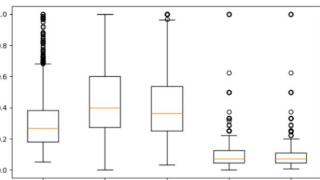
Round 2: second-best mutant from line A, line B, line C, ...

Round 3: third-best mutant from each line, ...

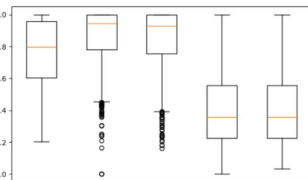
5. Budget 소진시 Stop.



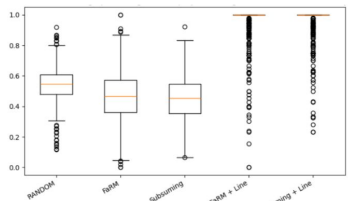
(a) 2% LineCov



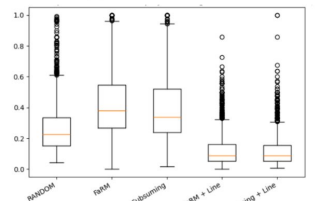
(b) 2% MaxShare



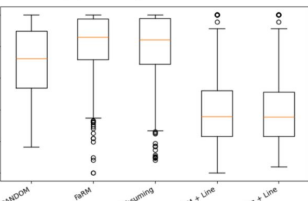
(c) 2% Top5Share



(a) 5% LineCov



(b) 5% MaxShare



(c) 5% Top5Share

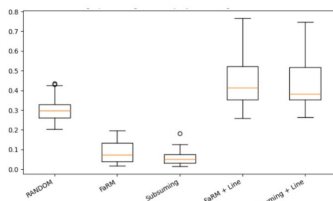
Structural Concentration

CoREBench

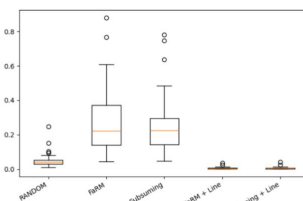
- Real regression faults: 45
- Project 수: 4
- 비교적 크고 실제로 개발, 유지보수되는 C project

CodeFlaws

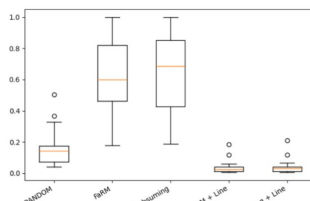
- Program 수: 1,692
- Generated mutants: 1,756,031
- Program 크기: 1-322 LOC
- 평균 크기: 약 36 LOC



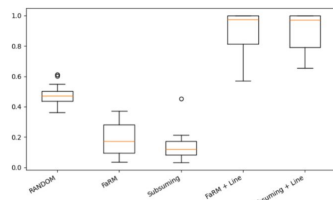
(a) 2% LineCov



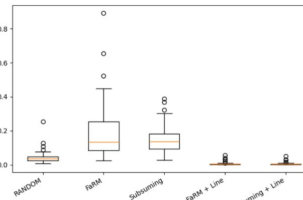
(b) 2% MaxShare



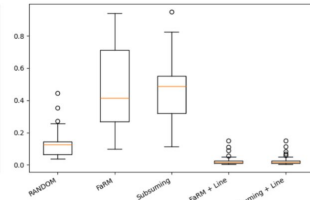
(c) 2% Top5Share



(a) 5% LineCov



(b) 5% MaxShare



(c) 5% Top5Share

Behavioral Redundancy & Fault Revelation Rate

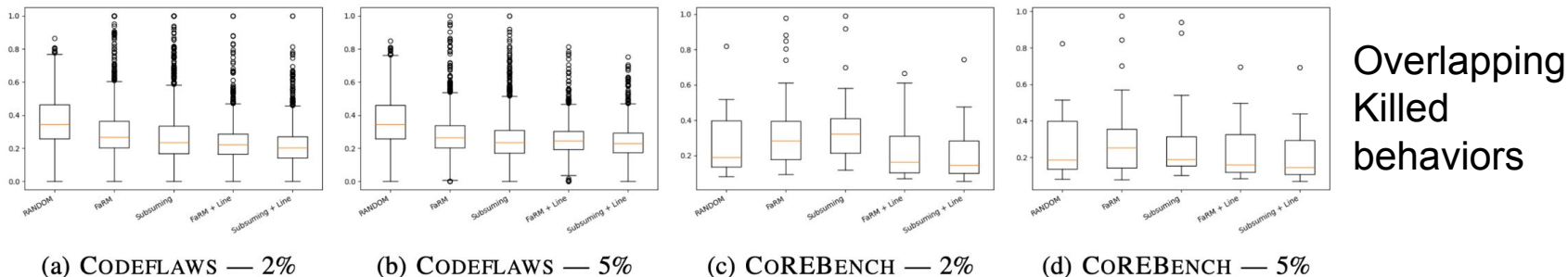


Fig. 3: Behavioral redundancy measured via average pairwise Jaccard similarity (lower is better). Orange lines are medians.

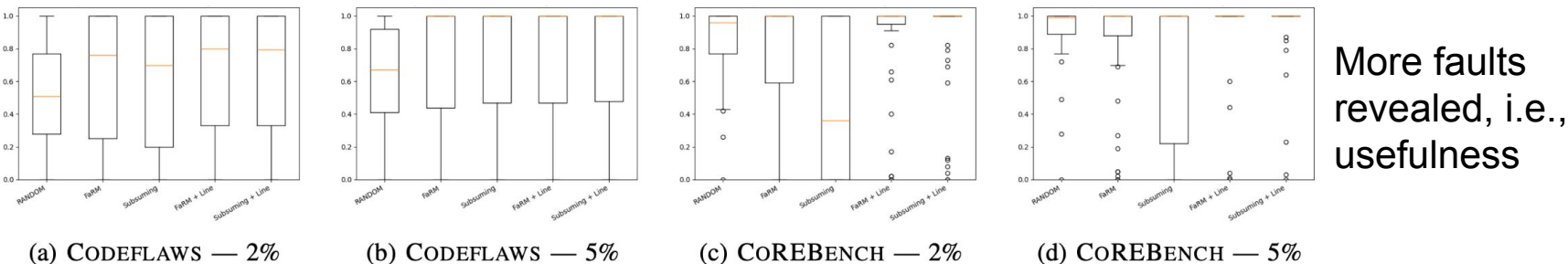


Fig. 4: Fault Revelation Rate (FRR) across datasets and selection budgets (higher is better). Orange lines are medians.

Alternative ranking signal sanity check

- Subsuming 또는 Fault Revelation 시그널에 specific?
- LLM-based scoring
 - DeepSeek-V3.2
 - program source code, mutant location, 기존 learning-based model에서 사용한 static mutant features
 - reachable & observable effect를 propagate & nearby mutants와 redundant X

TABLE II: LLM ablation on CODEFLAWS; FRR and Jaccard are means; others are medians.

Budget	Method	FRR	LineCov	MaxShare	Top5Share	Jaccard
2%	LLM	0.505	0.250	0.462	1.000	0.403
	LLM + Line	0.555	0.764	0.067	0.333	0.352
5%	LLM	0.619	0.357	0.396	0.929	0.395
	LLM + Line	0.669	1.000	0.062	0.303	0.338

Global top-k → structural concentration
→ higher redundancy

BlindSpot!: ranking만 믿으면 생기는 문제

Learning-based top-k는 좋은 mutant를 고르려 하지만,
같은 line/context에 budget을 몰아줄 수 있다.

**Structural
Concentration**



**Behavioral
Redundancy**



**FRR
Brittleness**

- 문제는 낮은 LineCov 자체보다, redundant kill behavior로 budget을 낭비할 수 있다는 점.
- LINEPOOL은 ranking을 버리지 않고 line-level diversity를 먼저 확보하는 guardrail.
- 한 줄 요약: 좋은 individual score $\neq$ 좋은 mutant set.

Longitudinal study of mutation testing

- 현재 뮤테이션 테스트: 한 시점의 `test suite`에 의해 `mutant`를 평가–Killed or Survived
- 그런데, 코드와 테스트는 계속 변한다.

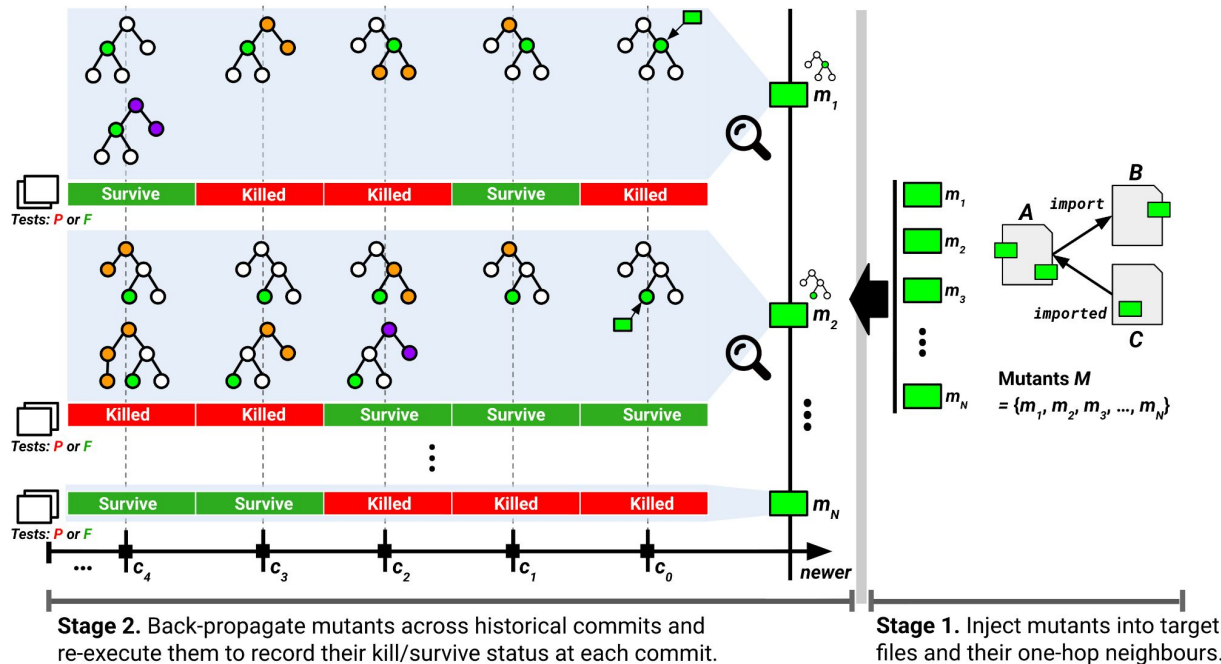
Longitudinal study of mutation testing

- 현재 뮤테이션 테스트: 한 시점의 **test suite**에 의해 **mutant**를 평가–Killed or Survived
- 그런데, 코드와 테스트는 계속 변한다.
- 그러면 **mutant**의 상태는?
 - Killed → Survived
 - Survived → Killed
 - Survived → Killed → Survived ...

Longitudinal study of mutation testing

- 현재 뮤테이션 테스트: 한 시점의 **test suite**에 의해 **mutant**를 평가–Killed or Survived
- 그런데, 코드와 테스트는 계속 변한다.
- 그러면 **mutant**의 상태는?
 - Killed → Survived
 - Survived → Killed
 - Survived → Killed → Survived ...
- Mutants as Evolving test-code interaction을 관찰하기 위한 longitudinal probe

Mutant Propagation



- Targeted mutation testing:
 - bug-fix로 수정된 file & 그 file을/이 import하는 one-hop neighbor
- Back-propagation
- BugsInPy

Project	#	# All	# Categ.
black	12	30244	26347
cookiecutter	1	211	186
fastapi	6	3213	2353
httpie	2	651	612
sanic	1	693	78
thefuck	24	4523	4377
tqdm	5	3307	2764
youtube-dl	14	32676	32378
Total	65	75518	69095

Mutant Propagation



Program evolution
(across commits)



Test evolution
(across commits)

Commit t (original)

```
34 if delta_t >= self.mininterval:
35     self.sp(format_meter(
36         self.n, self.total, cur_t-self.start_t, self.ncols,
37         self.prefix, self.ascii, self.unit, self.unit_scale))
38     if self.dynamic_mininters: # mutant: if not self.dynamic_mininters
39         self.mininters = max(self.mininters, delta_it)
40
41 self.last_print_n = self.n
42 self.last_print_t = cur_t
```

Propagated
(mutant line
unchanged)

Commit t' (evolved)

```
34 if delta_t >= self.mininterval:
35     if self.gui:
36         # GUI update branch added at t'
37         ...
38     else:
39         self.sp(format_meter(
40             self.n, self.total, elapsed, self.ncols,
41             self.desc, self.ascii, self.unit, self.unit_scale))
42     if self.dynamic_mininters: # mutant: if not self.dynamic_mininters
43         self.mininters = max(self.mininters, delta_it)
44 self.last_print_n = self.n
45 self.last_print_t = cur_t
```

i The program added
a GUI branch, but
our mutant line
is unchanged and
still reachable.

Commit t (original test)

```
def test_update():
    our_file = StringIO()
    progressbar = tqdm(total=3, file=our_file, mininters=1)
    # make 3 increments in total
    progressbar.update(2)

    progressbar.update(-10) # should default to +1
    our_file.seek(0)
    assert '| 2/3 ' in our_file.read()

    our_file.close()
```



Status at t: SURVIVED
(No assertion checks the output affected by mininters)



Commit t' (evolved test)

```
def test_update():
    our_file = StringIO()
    progressbar = tqdm(total=2, file=our_file, mininters=1)

    progressbar.update(2)
    our_file.seek(0)
    assert '| 2/2 ' in our_file.read()

    progressbar.desc = 'dynamically notify of 4 increments in total'
    progressbar.total = 4
    progressbar.update(-10) # should default to +1

    our_file.seek(0)
    assert '| 3/4 ' in our_file.read() # New assertion
    our_file.seek(0)
    assert 'dynamically notify of 4 increments in total' in our_file.read()
    our_file.close()
```



Status at t': KILLED
(New assertion detects the behavior affected by mininters)



New assertion checks
the later output that
is affected by mininters.
Mutant is now **KILLED**.



Mutant line (propagated)



New assertion targeting the mutant effect



Test evolved (oracle strengthened)



Program change is unrelated to the mutant

Mutant status를 어떻게 볼 것인가?

Pattern	Collapsed Sequence	Description
Stable Survived	S	The mutant consistently survives
Stable Killed	K	The mutant is consistently killed
Survived-to-Killed	$S \rightarrow K$	The mutant becomes killable over time
Killed-to-Survived	$K \rightarrow S$	The mutant ceases to be killable over time
Survived-Killed-Survived	$S \rightarrow K \rightarrow S$	The mutant is temporarily killable: e.g., $S, K, K, S, S, \dots, S$
Killed-Survived-Killed	$K \rightarrow S \rightarrow K$	The mutant temporarily ceases to be killable: e.g., $K, K, S, S, \dots, K, K$
Recurrent	≥ 3 transitions	The mutant changes status ≥ 3 times: e.g., $S, K, K, S, S, K, S, \dots, S$

- **RQ1. Pattern Prevalence:** 어떤 kill/survive pattern이 얼마나 자주 나타나는가?
- **RQ2. Region Distribution:** 그 pattern이 fix-context 근처에 몰리는가, target scope 전반에 퍼지는가?
- **RQ3. Evolution Context:** status stability/transition이 어떤 code/test change와 함께 나타나는가?

핵심은 “나중에 kill됨 = bug exposure”라고 바로 해석하지 않는 것.

Mutant status transition은 evolving test-code sensitivity를 보는 fine-grained signal에 가깝다.

Mutant status를 어떻게 볼 것인가?

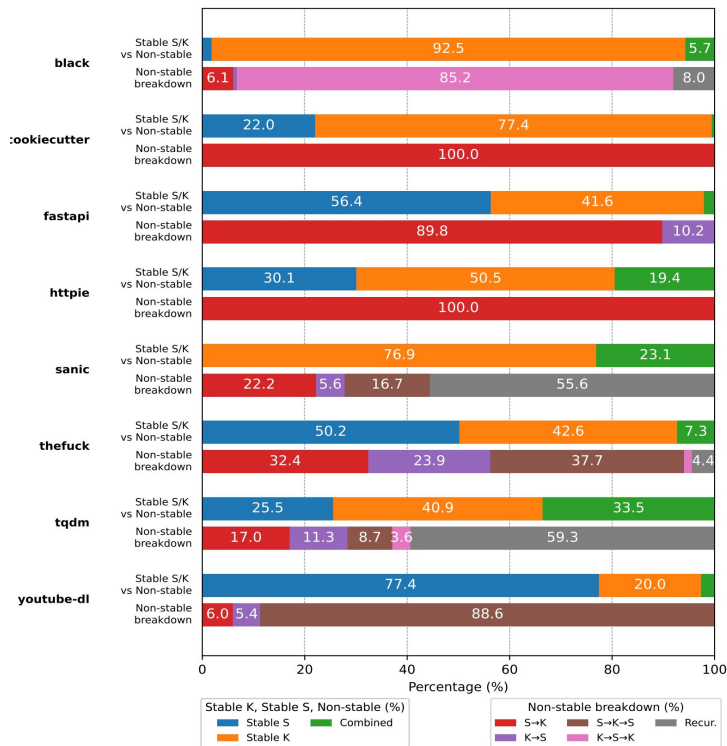
Pattern	Collapsed Sequence	Description
Stable Survived	S	The mutant consistently survives
Stable Killed	K	The mutant is consistently killed
Survived-to-Killed	$S \rightarrow K$	The mutant becomes killable over time
Killed-to-Survived	$K \rightarrow S$	The mutant ceases to be killable over time
Survived-Killed-Survived	$S \rightarrow K \rightarrow S$	The mutant is temporarily killable: e.g., $S, K, K, S, S, \dots, S$
Killed-Survived-Killed	$K \rightarrow S \rightarrow K$	The mutant temporarily ceases to be killable: e.g., $K, K, S, S, \dots, K, K$
Recurrent	≥ 3 transitions	The mutant changes status ≥ 3 times: e.g., $S, K, K, S, S, K, S, \dots, S$

- **RQ1. Pattern Prevalence:** 어떤 kill/survive pattern이 얼마나 자주 나타나는가?
- **RQ2. Region Distribution:** 그 pattern이 fix-context 근처에 몰리는가, target scope 전반에 퍼지는가?
- **RQ3. Evolution Context:** status stability/transition이 어떤 code/test change와 함께 나타나는가?

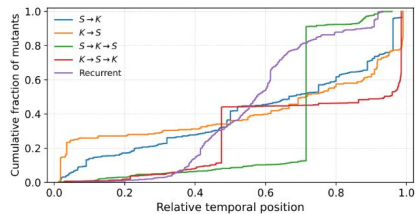
핵심은 “나중에 kill됨 = bug exposure”라고 바로 해석하지 않는 것.

Mutant status transition은 evolving test-code sensitivity를 보는 fine-grained signal에 가깝다.

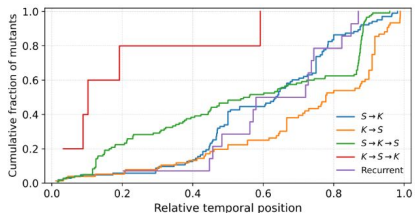
RQ1. Pattern Prevalence



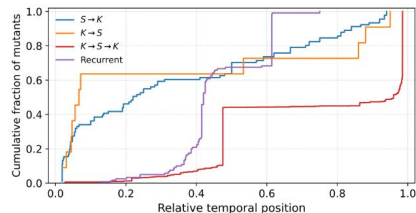
- 대다수가 stable: stable-survived/killed
- 그러나, status-changing 유턴트들 내부의 비율을 각 프로젝트마다 다름.



(a) All

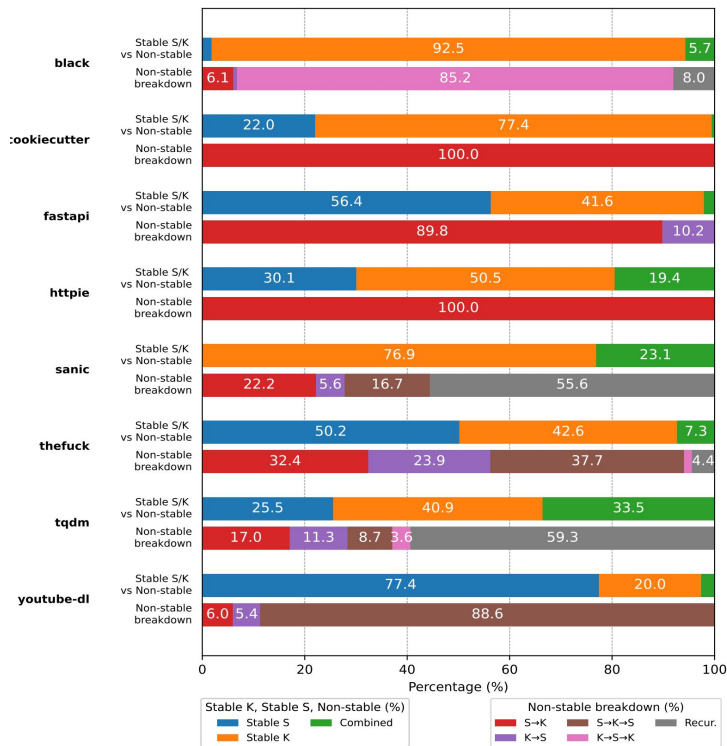


(b) thefuck



(c) black

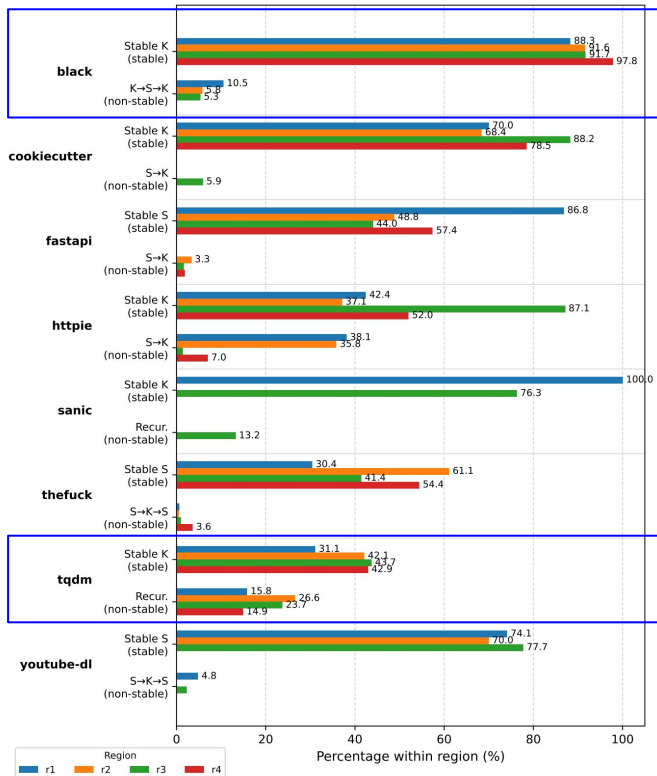
RQ1. Pattern Prevalence



- 대다수가 stable: stable-survived/killed
- 그러나, status-changing 유턴트들 내부의 비율을 각 프로젝트마다 다름.

- 그러면 이러한 유턴트들이 어디(region)에 분포할까? ⇒ *RQ2. Region Distribution*

RQ2. Region Distribution



“그 mutants가 fix와 가까운 곳에 몰려 있는가, 아니면 target scope 전체에 퍼져 있는가?”

- r1: a fix-modified method
- r2: a direct caller/callee of a fix-modified method
- r3: another method in a fix-modified file
- r4: a remaining method in an imported/importing one-hop neighbor file.

- **black**: non-stable pattern이 r1에서 높고 r4에서 사라짐 → localized tendency
- **tqdm**: recurrent pattern이 여러 region에 넓게 존재 → broad target-scope tendency
- **httpie**: S→K가 fix-related region에 상대적으로 집중

⇒ 일관된 단일 spatial pattern은 없고, project별로 heterogeneous한 regional profile을 보인다.

RQ3. Evolution Context

Project	Commit-pair group	# Pairs	Commit magnitude			Commit change type (%)		
			Size	Code Churn	Test Churn	Code-only	Test-only	Code+test
black	All stable	934	31 [11, 83]	8 [2, 34]	2 [0, 21]	43.6	7.0	49.4
	All transition	126	54 [23.2, 85]	12 [4.2, 38]	15 [0, 44.8]	27.0	9.5	64.0
	Only $S \rightarrow K$	73	63 [28, 114]	12 [6, 38]	20 [6, 81]	17.8	12.3	69.9
	Only $K \rightarrow S$	32	56.5 [20, 67.5]	33.5 [5.8, 38]	13.5 [1.5, 25.2]	25.0	3.1	71.9
	Mixed transition	21	31 [6, 75]	4 [2, 17]	0 [0, 6]	61.9	9.5	28.6
tqdm	All stable	162	36 [12.2, 128]	16.5 [4, 54.5]	0 [0, 18.8]	51.2	1.2	47.6
	All transition	87	99 [21, 277]	37 [13, 92]	9 [0, 63]	37.9	0.0	62.1
	Only $S \rightarrow K$	36	128.5 [23.8, 363.2]	56 [6, 153.2]	11 [0, 77]	33.3	0.0	66.7
	Only $K \rightarrow S$	21	96 [24, 127]	32 [15, 56]	8 [0, 44]	33.3	0.0	66.7
	Mixed transition	30	53 [18, 264.2]	20.5 [14, 105]	4.5 [0, 50.5]	46.7	0.0	53.3
thefuck	All stable	1969	49 [18, 110]	22 [8, 45]	19 [0, 55]	27.0	2.9	70.1
	All transition	164	46.5 [19, 124]	22.5 [9.8, 50]	15 [0, 58]	35.6	1.8	62.6
	Only $S \rightarrow K$	77	39 [16, 98]	18 [6, 33]	16 [0, 68]	29.0	4.0	67.1
	Only $K \rightarrow S$	70	63.5 [19, 150]	23 [11, 49]	15 [0, 59.8]	41.4	0.0	58.6
	Mixed transition	17	84 [32, 124]	40 [24, 121]	12 [0, 21]	41.2	0.0	58.8
Total	All stable	4762	32 [8, 90]	14 [4, 40]	0 [0, 28]	51.1	3.8	45.1
	All transition	413	58 [20, 143]	20 [6, 56]	12 [0, 54]	34.2	3.6	62.2
	Only $S \rightarrow K$	206	64.5 [23.2, 167]	19 [6, 57]	16 [0, 78.8]	25.8	5.8	68.3
	Only $K \rightarrow S$	134	62 [19, 120.5]	23 [10, 49]	10 [0, 45.8]	36.6	0.7	62.7
	Mixed transition	73	51 [18, 143]	20 [6, 90]	0 [0, 21]	53.5	2.7	43.8

Project	New test (%)		Updated test (%)		Existing test (%)	
	$S \rightarrow K$	Bug	$S \rightarrow K$	Bug	$S \rightarrow K$	Bug
black	0.4	90.0	84.8	10.0	14.8	0.0
cookiecutter	0.0	0.0	100.0	100.0	0.0	0.0
fastapi	1.7	80.0	0.8	0.0	97.5	20.0
httpie	100.0	16.7	0.0	16.7	0.0	66.7
sanic	0.0	100.0	0.0	0.0	100.0	0.0
thefuck	9.4	26.8	6.5	66.1	84.1	7.1
tqdm	12.6	20.0	4.4	40.0	82.9	40.0
youtube-dl	5.9	13.3	0.4	86.7	0.0	0.0
Total	3.4	35.5	64.2	51.8	27.3	12.7

Similar to bug-exposure?

All stable pairs vs All transition pairs

- Median commit size: 58 vs. 32
- Median code churn: 20 vs. 14
- Median test churn: 12 vs. 0
- Code+Test: 72.2 vs 45.1

각 transition pattern:

- $S \rightarrow K$, $K \rightarrow S$ vs Mixed transitions: 전자가 특히 code+test $\uparrow$ && test churn $\uparrow$

새로운 테스트의 등장보다는 기존 테스트로 인해 transition.

$\Rightarrow S \rightarrow K$ transition은 실제 bug가 새롭게 드러났다는 직접적 증거라기보다, evolving test-code interaction에서 test sensitivity가 바뀌었다는 signal.

RQ3. Evolution Context

transition이 “어떤 변화와 같이” 나타나는가?

Stable pair → Transition pair

Median commit size: 32 → 58

Median code churn: 14 → 20

Median test churn: 0 → 12

Code+Test change: 45.1% → 62.2%

해석

- Transition: 더 active한 code/test evolution
- S→K: new test보다 updated/existing test에서 자주 발생
- Mixed/Recurrent: 개선/악화보다 instability signal

정리: mutant-status transition은 direct bug-exposure proxy라기보다, test-code sensitivity가 바뀐 흔적으로 보는 편이 안전하다.

RQ3. Evolution Context

transition이 “어떤 변화와 같이” 나타나는가?

Stable pair → Transition pair

Median commit size: 32 → 58

Median code churn: 14 → 20

Median test churn: 0 → 12

Code+Test change: 45.1% → 62.2%

해석

- Transition: 더 active한 code/test evolution
- S→K: new test보다 updated/existing test에서 자주 발생
- Mixed/Recurrent: 개선/악화보다 instability signal

정리: mutant-status transition은 direct bug-exposure proxy라기보다, test-code sensitivity가 바뀐 흔적으로 보는 편이 안전하다.

결론.

Mutant는 그냥 “하나씩” 보면 안 된다.

Selection-time

- 개별 score가 높아도 set은 redundant할 수 있음
- Ranking quality + diversity가 같이 필요
- LINEPOOL = diversity guardrail

Evolution-time

- Killed/Survived는 snapshot label로 끝나지 않음
- Transition은 bug exposure보다 sensitivity signal
- Mixed/Recurrent는 아직 열린 문제

**즉, mutant usefulness는 static/individual property가 아니라,
set composition과 code-test evolution 속에서 결정된다.**